

GuardianIQ Engineering Handbook

Phase 4 / Step 4 Implementation Specification

Governance Event Architecture

End-to-End Developer Specification for Frontend, Backend, Data Model, APIs, Event Store, Audit Timeline, Outbox, Monitoring and QA

Field	Value
Platform	GuardianIQ
Phase / Step	Phase 4 / Step 4
Title	Governance Event Architecture - End-to-End Implementation Specification
Primary Audience	Product Owner, Project Manager, Backend Developer, Frontend Developer, Data Engineer, QA Engineer
Primary Source	GuardianIQ Step 4 - Governance Event Architecture Foundation
Prerequisites	Phase 0 Identity/Foundation, Phase 1 Registry, Phase 2 Workflow Scheduling, Phase 3 Relationship Architecture
Document Status	Developer implementation reference

Table of Contents

1. Executive Summary
2. Phase 4 Scope and Non-Scope
3. Fit with Previous GuardianIQ Phases
4. Target Event Architecture
5. Canonical Event Lifecycle
6. Frontend Implementation Specification
7. Backend Implementation Specification
8. Data Model and PostgreSQL Implementation Blueprint
9. API Specification and Contracts
10. Event Producers, Consumers and Routing
11. Audit Timeline and Traceability
12. Security, Privacy, Retention and Immutability
13. Observability and Monitoring
14. End-to-End Workflows
15. Developer Build Sequence
16. QA and Acceptance Criteria
17. Deliverables and Definition of Done
18. Appendices

1. Executive Summary

This document converts GuardianIQ Step 4 - Governance Event Architecture into an implementation-ready specification. The objective of Phase 4 is to make GuardianIQ event-first: every material governance activity must create a structured, immutable, traceable and searchable governance event. This allows the product to reconstruct lifecycle history, power audit timelines, support policy evidence, route notifications, detect SLA breaches and provide a trusted evidence backbone for AI governance.

The core implementation pattern is a hybrid architecture: operational tables continue to hold current state, while an append-only event store preserves historical truth. A durable outbox and dispatcher make event processing reliable. Event consumers build audit timelines, notifications, dashboard metrics, SLA monitoring and dead-letter review.

Phase 4 Capability	Developer Outcome	Primary Owner
Canonical Event Envelope	Shared schema for all governance events with actor, subject, correlation, policy, risk, payload and integrity metadata.	Backend / Data
Append-only Event Store	PostgreSQL tables for governance_events and supporting outbox, processing logs, retention rules and schema registry.	Aayush / Pankaj
Event Publisher Service	Reusable backend service called by registry, workflow, relationship, approval, policy and agent modules.	Pankaj
Audit Timeline APIs	Query APIs to reconstruct object and correlation timelines.	Pankaj / Aayush
Frontend Event Experience	Event Explorer, Subject Timeline, Correlation View, Dead Letter Review, Audit Export screens.	Poorvith
Monitoring and Reliability	Outbox retry, dead-letter handling, processing logs, event health dashboard.	Pankaj
QA Evidence	Tests proving event creation, immutability, correlation, access control, audit timeline reconstruction and failure recovery.	Pankaj / PM

Implementation principle:

- Do not update or delete governance event records through normal application paths.
- Every material state change in GuardianIQ must emit an event using the canonical envelope.
- Events must be searchable by tenant, event type, actor, subject, policy, date and correlation ID.
- Event payloads must be explicit but not overloaded with full documents, raw prompts or sensitive data.
- Audit timelines must be derived from events rather than recreated from mutable operational tables.

2. Phase 4 Scope and Non-Scope

2.1 In Scope

- Governance event data model and physical database design.
- Canonical event envelope and schema validation.
- Backend event module: publisher, repository, validation, outbox, dispatcher, consumer log and dead-letter handling.
- Event query APIs: search, event detail, subject history, correlation timeline, audit timeline and audit export.
- Frontend screens for Event Explorer, Event Detail Drawer, Subject Timeline, Correlation Timeline, Dead Letter Review and Audit Export.
- Integration of event publishing into existing GuardianIQ modules: identity, registry, relationship, workflow scheduling, agent execution and approvals.
- Security controls for event visibility using tenant_id, RBAC, ABAC, classification and retention policy.
- Monitoring metrics for event ingestion, outbox lag, consumer failures, dead-letter events and high-risk event trends.
- QA scenarios for event creation, immutability, correlation, retries, dead-letter recovery, audit reconstruction and security scope.

2.2 Not in Scope for Phase 4 MVP

- Full event sourcing for all operational state.
- Large-scale Kafka/EventBridge infrastructure unless required by deployment constraints.
- Advanced event replay across all modules in production.
- Full compliance package automation across every framework.
- Permanent graph database migration; PostgreSQL remains the primary system of record for this phase.

- Long-form raw AI prompt/output storage inside events. Events should reference artifacts instead.

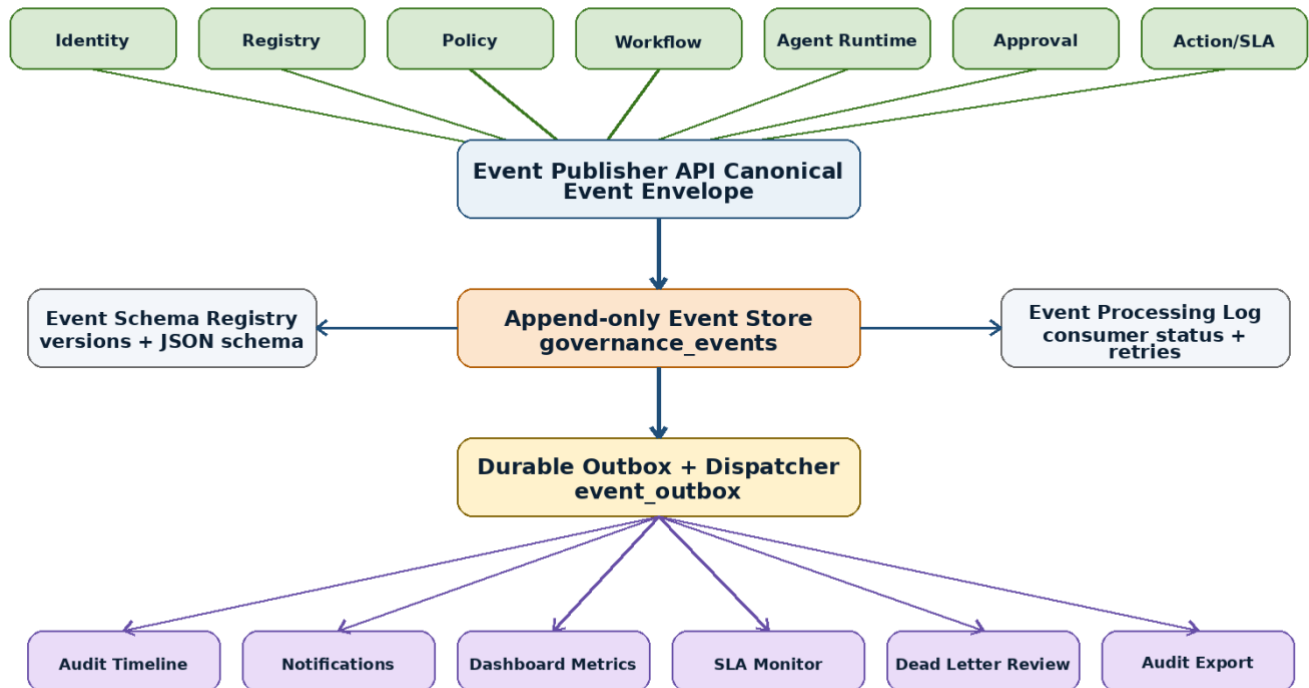
3. Fit with Previous GuardianIQ Phases

Phase 4 does not replace earlier phases. It makes them auditable and traceable.

Previous Phase	Existing Capability	Phase 4 Event Integration
Phase 0 - Identity	Users, roles, groups, departments, delegation and accountability.	Emit identity and delegation events; enrich all events with actor context.
Phase 1 - Registry	AI models, providers, agents, tools, workflows and data sources.	Emit registry lifecycle events such as AI_MODEL_REGISTERED and AGENT_PERMISSION_UPDATED.
Phase 2 - Workflow Scheduling	Schedules, workflow runs, agent assignments, boundaries and execution history.	Emit schedule/run/action events and correlate full run lifecycle.
Phase 3 - Relationship Architecture	Generic and specific relationship layer, validation, graph context.	Emit relationship lifecycle events and use relationship context in event payloads.
Phase 4 - Event Architecture	Event store, outbox, audit timeline, monitoring and event UI.	Provide immutable historical truth for all current and future modules.

4. Target Event Architecture

GuardianIQ Phase 4 - Governance Event Architecture



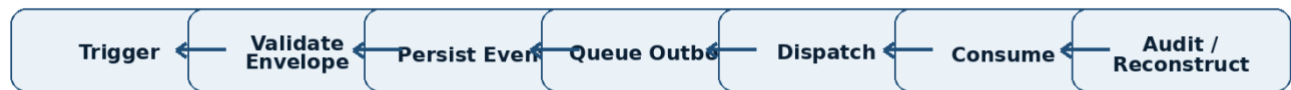
The architecture uses a central Event Publisher API and shared event module. Producers call the publisher synchronously inside business transactions. The event is persisted to governance_events and queued in event_outbox. A dispatcher then routes events to consumers such as audit timeline, notifications, dashboard metrics, SLA monitor and dead-letter review.

Architecture Component	Responsibility	Implementation Notes
Event Publisher API	Receives normalized event creation requests from backend modules.	FastAPI router/service; validates envelope; enriches actor context.
Governance Event Store	Append-only immutable table for canonical events.	PostgreSQL governance_events with JSONB contexts and indexed keys.

Event Outbox	Reliable queue for asynchronous dispatch.	Transactional outbox pattern; event saved before dispatch.
Dispatcher Job	Processes pending outbox rows and sends to internal consumers.	Background worker; retry and dead-letter handling.
Consumer Log	Records consumer status per event.	event_processing_log prevents silent failures.
Dead Letter Queue	Stores events that failed processing after retry threshold.	Requires admin review and retry action.
Audit Timeline Service	Builds subject and correlation timelines from events.	Can be query-time for MVP; derived read model later.
Frontend Event Screens	Allows users to search, inspect, export and troubleshoot events.	Event Explorer, Timeline, Dead Letter Review, Audit Export.

5. Canonical Event Lifecycle

Canonical Governance Event Lifecycle



The event must be persisted before downstream processing. Consumers may fail or retry without losing the immutable event record.

1. Trigger: a user action, AI action, policy evaluation, workflow transition, scheduled monitor or external integration creates an event-worthy occurrence.
2. Validate Envelope: the event producer constructs the canonical event envelope and validates required metadata.
3. Persist Event: the event is inserted into `governance_events` before downstream processing.
4. Queue Outbox: an outbox record is created transactionally with the event.
5. Dispatch: the background dispatcher publishes the event to internal consumers.
6. Consume: consumers update audit timelines, notifications, metrics, SLA state or integrations.
7. Monitor: failures, retries and dead-letter records are tracked.
8. Audit/Reconstruct: users reconstruct object lifecycle or correlation timelines from events.

5.1 Event Naming Rules

- Use UPPER_SNAKE_CASE.
- Use past-tense event names because events record what happened.
- Avoid vague events such as `OBJECT_UPDATED` where a specific event exists.
- Use stable event names; evolve payloads using `event_version`.
- Do not reuse one event type with multiple incompatible meanings.

Good Event Name	Avoid	Reason
AI_MODEL_REGISTERED	MODEL_SAVE	The good name clearly records completed business meaning.

POLICY_TRIGGERED	POLICY_CHECK	Triggered indicates a governance control fired.
APPROVAL_GRANTED	APPROVAL_DONE	Granted captures decision direction.
AGENT_ACTION_BLOCKED	ERROR	Blocked action is governance evidence, not a generic error.
AUDIT_PACKAGE_EXPORTED	EXPORT	Audit export has governance significance.

6. Frontend Implementation Specification

Phase 4 Frontend Screens - Event and Audit Experience



The Phase 4 frontend introduces event-first screens that allow users to search governance events, inspect payloads, reconstruct object timelines, understand correlation chains, monitor event processing failures and generate audit evidence packages.

6.1 Navigation Placement

Menu Area	Screen	Route	Purpose
Audit & Events	Event Explorer	/events	Search and inspect all permitted governance events.
Audit & Events	Event Detail	/events/{eventId}	View envelope, payload, actor, subject, correlation and integrity fields.
Audit & Events	Subject Timeline	/audit/timeline/{entityType}/{entityId}	Reconstruct lifecycle of a single object.
Audit & Events	Correlation Timeline	/events/correlation/{correlationId}	View full business transaction lifecycle.
Audit & Events	Dead Letter Review	/events/dead-letter	Review failed event processing and retry where allowed.
Audit & Events	Audit Export	/audit/export	Generate scoped audit packages.
Admin	Event Schema Registry	/admin/event-schemas	Manage event type/version metadata.
Admin	Retention Rules	/admin/event-retention	Configure retention policy by classification and event category.

6.2 Screen Specifications

Screen	Purpose	Core Components	Primary APIs	Developer Notes
Event Explorer	Allows audit, governance and admin users to search events by type, category, date, subject, actor, policy, risk level, source service and correlation ID.	EventSearchPage, EventFilterPanel, EventDataGrid, EventDetailDrawer, ExportButton	GET /api/events	Must support pagination, saved filters later, role-based column visibility and masked payloads.
Event Detail	Shows the full canonical	EventEnvelopePanel,	GET /api/events/{event_id}	Use JSON viewer with

Drawer/Page	envelope, payload, policy/risk context, integrity fields, related events and processing status.	JsonPayloadViewer, RelatedEventsList, ProcessingStatusPanel		copy disabled for sensitive fields where required.
Subject Timeline	Shows chronological lifecycle of one object, such as AI Agent, Workflow, Recommendation, Approval or Action.	TimelineHeader, TimelineFilters, TimelineList, TimelineEventCard, EvidenceLinks	GET /api/events/subject/{entity_type}/{entity_id}	Must group related events and show actor, time, state change and rationale.
Correlation Timeline	Shows end-to-end lifecycle across recommendation, risk, policy, approval, action and audit events.	CorrelationSummary, ChainTimeline, CausationGraph, EventCard	GET /api/events/correlation/{correlation_id}	Use same correlation_id across the chain; support jump to subject.
Dead Letter Review	Allows platform/admin users to inspect failed event processing records and retry or mark resolved.	DeadLetterGrid, FailureDetailPanel, RetryActionModal	GET /api/events/dead-letter; POST /api/events/dead-letter/{id}/retry	Retry should be permission controlled and audited.
Audit Export	Allows auditor/compliance role to generate event-backed audit package for selected scope.	ExportScopeForm, PreviewSummary, ExportHistoryGrid	POST /api/audit/export; GET /api/audit/export/{id}	Exports must log requested_by, scope, timestamp and hash.
Event Schema Registry	Admin screen to manage event type versions and schema status.	SchemaList, SchemaDetail, SchemaJsonViewer	GET /api/event-schemas	Schema changes should be controlled and not break existing events.
Retention Rules	Admin screen to manage retention class by event category and classification.	RetentionRuleGrid, RetentionRuleForm	GET/PUT /api/event-retention-rules	Retention rule updates must be audited.

6.3 Event Explorer Field Specification

Field / Filter	Type	Required?	Behavior	API Parameter
Date Range	DateTime range	Optional	Defaults to last 7 days for performance. Required for export over large datasets.	from, to
Event Type	Multi-select enum	Optional	Loads from event schema registry / event types endpoint.	event_type
Event Category	Multi-select enum	Optional	Identity, Registry, Policy, Risk, Recommendation, Approval, Agent, Violation, Audit.	category
Subject Type	Enum	Optional	AI_MODEL, AI_AGENT, WORKFLOW, RECOMMENDATION, APPROVAL, ACTION, POLICY, TOOL, DATA_SOURCE.	subject_type
Subject ID	Text	Optional	Exact match or begins-with depending API support.	subject_id
Actor	Search select	Optional	User/agent/system actor lookup.	actor_id
Correlation ID	Text	Optional	Exact correlation timeline lookup.	correlation_id
Risk Level	Multi-select	Optional	LOW/MEDIUM/HIGH/CRITICAL.	risk_level
Source Service	Multi-select	Optional	policy-engine, approval-service, agent-runtime, registry-service, workflow-service.	source_service
Classification	Multi-select	Optional	PUBLIC, INTERNAL, CONFIDENTIAL, RESTRICTED.	classification

6.4 UI State and Error Handling

State	Expected UI Behavior	Developer Requirement
Loading	Show skeleton rows and disable export/retry actions.	Do not show stale event results as current.
Empty	Display message: No governance events found for selected filters.	Offer reset filters.
Permission Denied	Show scoped error without revealing hidden event existence.	Return 403 from API and avoid payload leakage.
Large Result Set	Ask user to narrow date range or export asynchronously.	Enforce server-side pagination and date limits.
Dead Letter Retry Failed	Show retry failure reason and keep record unresolved.	Log retry attempt event.
Payload Masked	Show masked sections with reason and permission requirement.	Do not render raw sensitive JSON for unauthorized users.

7. Backend Implementation Specification

7.1 Backend Module Structure

backend/app/modules/events/

```
models.py           # SQLAlchemy models for governance_events, event_outbox, processing logs
schemas.py          # Pydantic event envelope and API models
event_types.py       # Event type/category enums and registry constants
```

```

repository.py      # Database persistence and query patterns
service.py         # EventPublisherService and EventQueryService
validators.py      # Canonical envelope validation
dispatcher.py      # Outbox dispatcher job
consumers.py       # Internal consumer orchestration
security.py        # Event visibility rules and payload masking
router.py          # FastAPI routes

```

```

backend/app/modules/audit/
  timeline_service.py # Subject and correlation timeline reconstruction
  export_service.py   # Audit package export

```

```

backend/app/shared/
  actor_context.py    # Current user/agent/system context enrichment
  correlation.py       # Correlation id generation and propagation
  event_integrity.py   # Event hash and chain-of-custody helpers

```

7.2 Backend Services

Service	Responsibilities	Key Methods
EventPublisherService	Validates and persists events; creates outbox record; returns event_id.	publish_event, enrich_event, validate_event, append_event
EventQueryService	Searches events by filter, actor, subject, policy, risk and correlation.	search_events, get_event, get_subject_events, get_correlation_events
EventDispatcherService	Processes event_outbox records, routes to consumers, records processing status and handles retries.	dispatch_pending, dispatch_one, mark_processed, mark_failed, dead_letter
AuditTimelineService	Builds object and correlation timeline responses for UI and export.	build_subject_timeline, build_correlation_timeline
EventSchemaService	Maintains event type/version registry and validates payload schema.	get_schema, validate_payload_schema, list_event_types
EventSecurityService	Applies tenant, RBAC, ABAC and classification visibility rules.	can_view_event, mask_payload, filter_events_by_scope
EventIntegrityService	Generates event hashes and export hashes.	hash_event, verify_event, hash_export_package
AuditExportService	Creates export packages and records export log entries.	create_export, get_export_status, generate_manifest

7.3 Event Publishing Rules

- Publish events inside the same business transaction when possible. If the primary action succeeds but the event insert fails, the transaction should fail for material governance operations.
- Use system actor context for scheduled jobs and dispatcher-created operational events.
- Every event must include tenant_id, event_type, event_version, occurred_at, recorded_at, source_service, actor, subject, correlation_id, payload, classification and retention_class.
- If a service receives an existing correlation_id from a previous event or API request, it must propagate it.
- If an event is caused by another event, causation_id must point to the parent event_id.
- If an event contains sensitive information, payload fields must be masked or referenced by artifact_id instead of storing raw content.

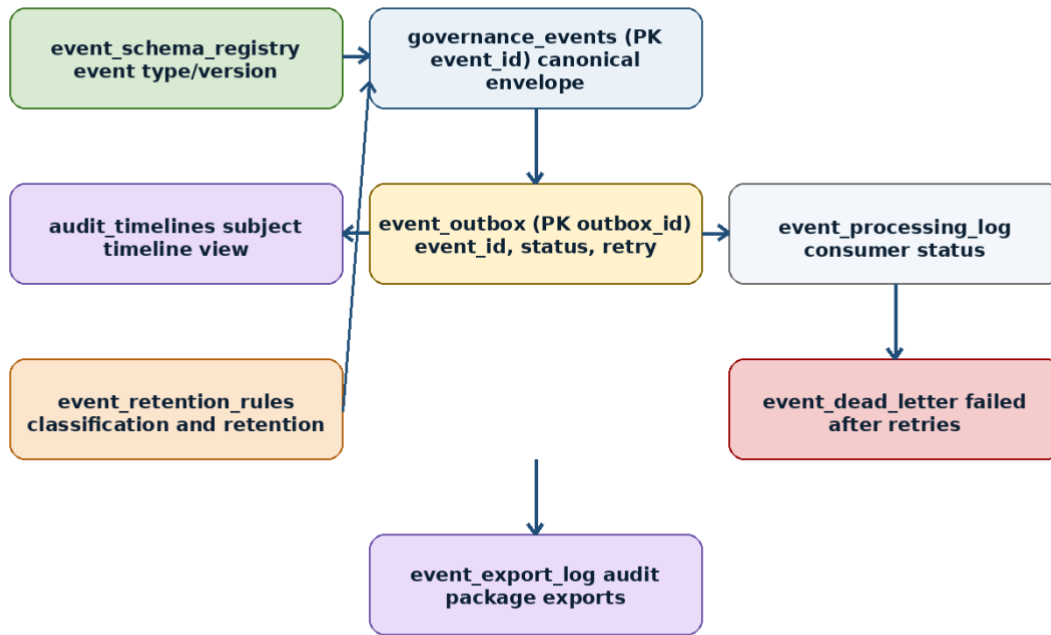
7.4 Producer Integration Points

Producer	Events	Trigger
Identity Service	USER_CREATED, ROLE_ASSIGNED, DELEGATION_STARTED, DELEGATION_ENDED	On user/role/delegation state changes.
Registry Service	AI_MODEL_REGISTERED, AI_MODEL_UPDATED, AGENT_REGISTERED, TOOL_REGISTERED, WORKFLOW_REGISTERED	On creation, update, activation, retirement and permission changes.
Relationship Service	RELATIONSHIP_CREATED, RELATIONSHIP_REVOKED, RELATIONSHIP_VALIDATION_FAILED	From Phase 3 relationship lifecycle operations.
Workflow/Scheduler Service	WORKFLOW_SCHEDULE_CREATED, WORKFLOW_RUN_STARTED, WORKFLOW_RUN_COMPLETED, WORKFLOW_RUN_FAILED	During schedule/run lifecycle.
Policy Engine	POLICY_EVALUATED, POLICY_TRIGGERED, POLICY_VIOLATION_DETECTED	During policy evaluation and enforcement.
Approval Service	APPROVAL_REQUESTED, APPROVAL_GRANTED,	During human-in-the-loop review.

	APPROVAL_REJECTED, ESCALATION_TRIGGERED	
Agent Runtime	AGENT_ACTION_REQUESTED, TOOL_ACCESS_ATTEMPTED, AGENT_ACTION_BLOCKED, AGENT_ACTION_EXECUTED	Before and after agent boundary enforcement.
Audit Export Service	AUDIT_TIMELINE_VIEWED, AUDIT_PACKAGE_EXPORTED	When audit timelines are viewed or exported.

8. Data Model and PostgreSQL Implementation Blueprint

Phase 4 Data Model - Event Store and Audit Reliability



Rule: governance_events is append-only. Operational tables keep current state; audit/analytics read models are derived from event_outbox.

The following data model is the recommended Phase 4 physical design. Exact schema names can be aligned with the existing GuardianIQ database conventions.

8.1 Required Tables

Table	Purpose	Key Columns
governance_events	Canonical immutable event store.	event_id, tenant_id, event_type, event_version, occurred_at, recorded_at, source_service, actor_json, subject_json, correlation_id, causation_id, risk_context_json, policy_context_json, payload_json, classification, retention_class, event_hash, previous_event_hash
event_outbox	Transactional outbox for reliable event dispatch.	outbox_id, event_id, tenant_id, topic, status, retry_count, next_retry_at, last_error, created_at, dispatched_at
event_processing_log	Tracks consumer processing status for each event.	processing_id, event_id, consumer_name, status, started_at, processed_at, retry_count, error_message
event_dead_letter	Stores failed event dispatch/consumer processing after retry threshold.	dead_letter_id, event_id, tenant_id, failure_reason, payload_snapshot_json, retry_count, status, created_at, resolved_at
audit_timelines	Optional derived timeline read model for performance.	timeline_id, tenant_id, subject_type, subject_id, event_sequence_json, last_event_at, updated_at
event_schema_registry	Registry of event types and schema versions.	schema_id, event_type, category, event_version, json_schema, status, effective_from, effective_to
event_retention_rules	Retention rules by event category/classification.	rule_id, event_category, classification, retention_class, retention_days, storage_tier, status
event_export_log	Records audit package export requests and integrity metadata.	export_id, tenant_id, requested_by, scope_json, file_reference, event_count, export_hash, status, created_at

8.2 Detailed Column Specification - governance_events

Column	Type	Constraint	Description
event_id	UUID	PK	Immutable event identifier.
tenant_id	UUID	NOT NULL	Tenant boundary; included in every query.
event_type	VARCHAR(100)	NOT NULL	Stable event name.
event_category	VARCHAR(80)	NOT NULL	Identity, Registry, Policy, Approval, Agent, Audit etc.
event_version	VARCHAR(20)	NOT NULL	Schema version for payload interpretation.
occurred_at	TIMESTAMPTZ	NOT NULL	When the event actually occurred.
recorded_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	When GuardianIQ recorded it.
source_service	VARCHAR(100)	NOT NULL	Service/module producing event.
source_system	VARCHAR(100)	NOT NULL	GuardianIQ or external system.
actor_json	JSONB	NOT NULL	Actor type, id, role, delegation context.
subject_json	JSONB	NOT NULL	Primary entity type/id/version.
correlation_id	VARCHAR(150)	NOT NULL	End-to-end flow identifier.
causation_id	UUID	NULL	Parent event that caused this event.
risk_context_json	JSONB	NULL	Risk score/level/context.
policy_context_json	JSONB	NULL	Policy ids, decision, violations.
payload_json	JSONB	NOT NULL	Event-specific payload.
classification	VARCHAR(50)	NOT NULL	Data sensitivity classification.
retention_class	VARCHAR(80)	NOT NULL	Retention policy reference.
event_hash	VARCHAR(256)	NULL	Optional integrity hash.
previous_event_hash	VARCHAR(256)	NULL	Optional hash chain.
created_at	TIMESTAMPTZ	NOT NULL DEFAULT now()	Database insertion timestamp.

8.3 Recommended Indexes

Index	Columns / Expression	Purpose
idx_events_tenant_time	tenant_id, occurred_at DESC	Default event search by tenant and date.
idx_events_type_time	tenant_id, event_type, occurred_at DESC	Filter event type quickly.
idx_events_category_time	tenant_id, event_category, occurred_at DESC	Category dashboard metrics.
idx_events_correlation	tenant_id, correlation_id, occurred_at	Correlation timeline reconstruction.
idx_events_subject_gin	GIN(subject_json)	Subject-based search when entity type/id stored in JSON.
idx_events_actor_gin	GIN(actor_json)	Actor-based search.
idx_events_payload_gin	GIN(payload_json)	Selective payload search for admin/debug only.
idx_outbox_status_retry	status, next_retry_at	Dispatcher queue scan.
idx_processing_event_consumer	event_id, consumer_name	Idempotent consumer processing.
idx_dead_letter_status	tenant_id, status, created_at DESC	Dead letter review screen.

8.4 SQL Skeleton

```
CREATE TABLE governance_events (
  event_id UUID PRIMARY KEY,
  tenant_id UUID NOT NULL,
  event_type VARCHAR(100) NOT NULL,
  event_category VARCHAR(80) NOT NULL,
  event_version VARCHAR(20) NOT NULL,
  occurred_at TIMESTAMPTZ NOT NULL,
  recorded_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  source_service VARCHAR(100) NOT NULL,
  source_system VARCHAR(100) NOT NULL DEFAULT 'GuardianIQ',
  actor_json JSONB NOT NULL,
  subject_json JSONB NOT NULL,
  correlation_id VARCHAR(150) NOT NULL,
  causation_id UUID NULL,
  risk_context_json JSONB NULL,
  policy_context_json JSONB NULL,
  payload_json JSONB NOT NULL,
  classification VARCHAR(50) NOT NULL,
  retention_class VARCHAR(80) NOT NULL,
  event_hash VARCHAR(256) NULL,
  previous_event_hash VARCHAR(256) NULL,
```

```
created_at TIMESTAMPTZ NOT NULL DEFAULT now()
```

```
);
```

```
CREATE TABLE event_outbox (
  outbox_id UUID PRIMARY KEY,
  event_id UUID NOT NULL REFERENCES governance_events(event_id),
  tenant_id UUID NOT NULL,
  topic VARCHAR(150) NOT NULL,
  status VARCHAR(50) NOT NULL DEFAULT 'PENDING',
  retry_count INTEGER NOT NULL DEFAULT 0,
  next_retry_at TIMESTAMPTZ NULL,
  last_error TEXT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  dispatched_at TIMESTAMPTZ NULL
);
```

```
CREATE INDEX idx_events_tenant_time ON governance_events (tenant_id, occurred_at DESC);
CREATE INDEX idx_events_type_time ON governance_events (tenant_id, event_type, occurred_at DESC);
CREATE INDEX idx_events_correlation ON governance_events (tenant_id, correlation_id, occurred_at);
CREATE INDEX idx_events_subject_gin ON governance_events USING GIN (subject_json);
CREATE INDEX idx_events_actor_gin ON governance_events USING GIN (actor_json);
CREATE INDEX idx_outbox_status_retry ON event_outbox (status, next_retry_at);
```

9. API Specification and Contracts

9.1 Standard Response Envelope

```
{
  "success": true,
  "data": {},
  "error": null,
  "request_id": "REQ-20260708-0001",
  "timestamp": "2026-07-08T09:00:00+05:30"
}
```

9.2 API Catalogue

Method	Endpoint	Purpose	Permission	Notes
POST	/api/v1/events	Publish event internally	CREATE_EVENT	Backend services only / admin limited
GET	/api/v1/events	Search events	VIEW_EVENTS	Supports filters and pagination
GET	/api/v1/events/{event_id}	Get event detail	VIEW_EVENTS	Payload may be masked
GET	/api/v1/events/subject/{entity_type}/{entity_id}	Get subject event history	VIEW_AUDIT_TIMELINE	Chronological lifecycle
GET	/api/v1/events/correlation/{correlation_id}	Get correlation timeline	VIEW_AUDIT_TIMELINE	End-to-end flow
POST	/api/v1/events/replay	Replay events to rebuild read models	ADMIN_EVENT_REPLAY	Controlled environments initially
GET	/api/v1/events/dead-letter	List failed processing records	VIEW_DEAD_LETTER	Admin/platform users
POST	/api/v1/events/dead-letter/{id}/retry	Retry dead-letter event	RETRY_DEAD_LETTER	Audited action
GET	/api/v1/audit/timeline/{entity_type}/{entity_id}	Audit-friendly subject timeline	VIEW_AUDIT_TIMELINE	UI timeline endpoint
POST	/api/v1/audit/export	Create audit package export	EXPORT_AUDIT_PACKAGE	Generates event export log
GET	/api/v1/event-schemas	List event schemas	VIEW_REFERENCE_DATA	Admin/reference
PUT	/api/v1/event-schemas/{schema_id}	Update event schema	MANAGE_EVENT_SCHEMA	Version controlled
GET	/api/v1/event-retention-rules	List retention rules	VIEW_REFERENCE_DATA	Admin/reference
PUT	/api/v1/event-retention-rules/{rule_id}	Update retention rule	MANAGE_RETENTION_RULES	Audited

9.3 Publish Event Request Example

```
{
  "event_type": "POLICY_TRIGGERED",
  "event_version": "1.0",
  "occurred_at": "2026-07-08T10:30:45+05:30",
  "source_service": "policy-engine",
  "source_system": "GuardianIQ",
  "subject": {
    "entity_type": "RECOMMENDATION",
    "entity_id": "REC-1044",
    "entity_version": "1"
  },
  "correlation_id": "CORR-REC-1044",
  "causation_id": "EVT-20260708-000001",
  "risk_context": {
    "risk_score": 92,
    "risk_level": "HIGH"
  },
  "policy_context": {
    "policy_ids": ["POL-007"],
    "policy_decision": "APPROVAL_REQUIRED"
  },
  "payload": {
    "trigger_reason": "risk_score >= 80",
    "approval_role_required": "RISK_MANAGER"
  },
  "classification": "INTERNAL",
  "retention_class": "AUDIT_7_YEARS"
}
```

9.4 Event Search Query Parameters

Parameter	Type	Required	Description
from	datetime	Recommended	Start date/time. Required for large searches.
to	datetime	Recommended	End date/time.
event_type	string/list	No	One or more event types.
event_category	string/list	No	Category filter.
subject_type	string	No	Subject entity type.
subject_id	string	No	Subject entity id.
actor_id	string	No	Actor filter.
correlation_id	string	No	Exact correlation chain.
risk_level	string/list	No	LOW/MEDIUM/HIGH/CRITICAL.
classification	string/list	No	Sensitivity classification.
page	int	No	Default 1.
page_size	int	No	Default 25, max 100.

10. Event Producers, Consumers and Routing

10.1 Topics / Channels

Topic	Purpose	Example Events
guardian.identity	Identity/access/delegation events.	USER_CREATED, ROLE_ASSIGNED, DELEGATION_STARTED
guardian.registry	AI model, agent, tool, workflow and data-source lifecycle.	AI_MODEL_REGISTERED, AGENT_REGISTERED
guardian.relationship	Relationship lifecycle and validation.	RELATIONSHIP_CREATED, RELATIONSHIP_REVOKED
guardian.workflow	Schedule and workflow run lifecycle.	WORKFLOW_RUN_STARTED, WORKFLOW_RUN_COMPLETED
guardian.policy	Policy lifecycle and policy evaluation.	POLICY_EVALUATED, POLICY_TRIGGERED
guardian.approval	Approval and escalation lifecycle.	APPROVAL_REQUESTED, APPROVAL_GRANTED

guardian.action	Action execution and outcome.	ACTION_CREATED, ACTION_EXECUTED, ACTION_FAILED
guardian.agent	AI agent tool access and action control.	AGENT_ACTION_BLOCKED, TOOL_ACCESS_ATTEMPTED
guardian.audit	Timeline, evidence and audit export events.	AUDIT_TIMELINE_VIEWED, AUDIT_PACKAGE_EXPORTED
guardian.violation	Policy and control failures.	POLICY_VIOLATION_DETECTED, SLA_BREACHED

10.2 Consumer Processing Rules

- Consumers must be idempotent. Reprocessing the same event should not duplicate timeline entries or notifications.
- Every consumer must write a processing log row with status SUCCESS, FAILED or SKIPPED.
- Transient failures should retry using exponential backoff or scheduled retry_at.
- After the retry threshold, the event should be moved to dead-letter review.
- Consumer errors must not mutate the original event.
- Dead-letter retry must be explicit, permission-controlled and audited.

11. Audit Timeline and Traceability

The audit timeline is the primary user-facing proof that GuardianIQ can reconstruct lifecycle history. It should be available from major object detail pages and the Audit & Events module.

11.1 Timeline Data Model

Timeline Element	Source	UI Display
Event Time	occurred_at	Chronological marker.
Actor	actor_json	User/agent/system identity and role.
Subject	subject_json	Object affected by event.
Event Type	event_type	Human-readable label and category badge.
State Change	payload_json.before / after or event-specific status fields	Previous and new state when applicable.
Policy Context	policy_context_json	Policy badge and decision.
Risk Context	risk_context_json	Risk level badge and score.
Evidence References	payload/evidence references	Links to artifacts when allowed.
Correlation	correlation_id	View full lifecycle chain.
Integrity	event_hash/export_hash	Shown in audit export package.

11.2 Correlation Example

```

CORR-REC-1044
RECOMMENDATION_CREATED
-> POLICY_EVALUATED
-> RISK_EVALUATED
-> POLICY_TRIGGERED
-> APPROVAL_REQUESTED
-> APPROVAL_GRANTED
-> ACTION_CREATED
-> ACTION_EXECUTED
-> AUDIT_TIMELINE_UPDATED

```

12. Security, Privacy, Retention and Immutability

Control Area	Requirement	Implementation Detail
Tenant Isolation	Every event must include tenant_id.	All queries must filter by tenant_id; cross-tenant event reads blocked.
RBAC	Users need permissions to view, export, retry or manage events.	VIEW_EVENTS, VIEW_AUDIT_TIMELINE, EXPORT_AUDIT_PACKAGE, RETRY_DEAD_LETTER.
ABAC	Event visibility depends on subject sensitivity, department/business unit, actor scope and classification.	EventSecurityService applies scope filter and payload masking.
Sensitive Payloads	Avoid raw PII, secrets, full documents or raw prompts in events.	Store references to artifact/evidence records.
Immutability	Events cannot be updated/deleted by application APIs.	Use append-only permissions; corrections as new EVENT_CORRECTION_RECORDED event.
Retention	Retention_class must be populated for every event.	event_retention_rules maps classification/category to retention_days

		and storage_tier.
Export Integrity	Audit exports must include export hash and event count.	event_export_log records requested_by, scope_json, file_reference and export_hash.

13. Observability and Monitoring

Metric / Alert	Why It Matters	Owner
Event ingestion rate by category	Confirms event publishing works and highlights behavior trends.	Platform / PM
Event validation failure count	Shows producer bugs or schema mismatch.	Backend
Outbox pending count and lag	Detects dispatch pipeline failure.	Backend / DevOps
Consumer processing failure count	Detects timeline/notification/dashboard failures.	Backend
Dead-letter count	Requires admin attention and retry.	Pankaj
High-risk event trend	Highlights policy/risk issues.	Governance Owner
Agent blocked action count	Shows guardrail enforcement and potential agent misuse.	AI Governance Owner
Audit export failures	Compliance evidence risk.	Admin / Compliance
Event store growth	Capacity and retention planning.	DB Owner

14. End-to-End Workflows

14.1 Registry Object Lifecycle Event Flow

9. User creates AI Model in registry.
10. Registry service saves operational record.
11. Registry service publishes AI_MODEL_REGISTERED event.
12. Event is inserted into governance_events and event_outbox.
13. Dispatcher processes event for audit timeline and dashboard metrics.
14. AI Model detail page shows event in object timeline.

14.2 Agent Action Blocked Flow

15. AI Agent requests action against tool/API.
16. Agent runtime emits AGENT_ACTION_REQUESTED.
17. Boundary and policy checks evaluate relationship/tool permissions.
18. Policy service emits POLICY_EVALUATED and possibly POLICY_TRIGGERED.
19. If blocked, runtime emits AGENT_ACTION_BLOCKED and POLICY_VIOLATION_DETECTED.
20. Notification consumer routes alert to owner/governance admin.
21. Audit timeline shows full sequence under same correlation_id.

14.3 Approval Lifecycle Flow

22. Recommendation triggers approval requirement.
23. Approval service emits APPROVAL_REQUESTED.
24. Reviewer opens approval and emits APPROVAL_REVIEW_STARTED.
25. Reviewer approves/rejects and emits APPROVAL_GRANTED or APPROVAL_REJECTED.
26. Action service emits ACTION_CREATED if approved.
27. Audit timeline reconstructs reviewer, decision, rationale and SLA status.

14.4 Dead Letter Recovery Flow

28. Dispatcher attempts to process outbox event.
29. Consumer fails due to transient or permanent error.
30. Processing log records failure and retry_count.
31. After threshold, event is moved to event_dead_letter.
32. Admin reviews failure in Dead Letter Review UI.
33. Admin retries or marks resolved; action emits DEAD_LETTER_RETRIED or DEAD_LETTER_RESOLVED event.

14.5 Audit Export Flow

34. Auditor selects scope: subject, date range, event types or correlation.
35. System validates permission and scope.
36. AuditExportService queries immutable events.
37. Export package is generated with manifest, event count and hash.
38. AUDIT_PACKAGE_EXPORTED event is generated.
39. event_export_log stores file_reference and export_hash.

15. Developer Build Sequence

Sequence	Workstream	Activity	Owner	Deliverable
1	Database	Create Phase 4 event tables, indexes and seed data.	Aayush	Migration scripts
2	Backend	Create event module models, schemas, repository and validations.	Pankaj	Events backend foundation
3	Backend	Implement EventPublisherService and integrate with existing modules.	Pankaj	Reusable event publishing
4	Backend	Implement outbox dispatcher, processing log and dead-letter handling.	Pankaj	Reliable event pipeline
5	Backend	Implement event search, detail, subject timeline and correlation timeline APIs.	Pankaj / Aayush	API layer
6	Frontend	Build Event Explorer, Event Detail Drawer, Subject Timeline and Correlation Timeline.	Poorvith	Core event UI
7	Frontend	Build Dead Letter Review and Audit Export screens.	Poorvith	Admin/audit UI
8	Integration	Wire event publishing into registry, relationship, workflow and approval flows.	Aayush / Pankaj	End-to-end event generation
9	QA	Execute functional, security, immutability, retry and timeline tests.	Pankaj	QA evidence
10	PM	Conduct final walkthrough and sign-off.	Project Manager	Phase 4 acceptance

16. QA and Acceptance Criteria

16.1 Test Matrix

Test ID	Area	Scenario	Expected Result
EVT-QA-001	Schema Validation	Publish event missing tenant_id.	API rejects event with field-level error.
EVT-QA-002	Schema Validation	Publish event with invalid event_type.	API rejects or flags unknown event type.
EVT-QA-003	Event Store	Create event through registry object creation.	governance_events row created.
EVT-QA-004	Outbox	Create event and verify outbox row.	event_outbox has PENDING or DISPATCHED record.
EVT-QA-005	Dispatcher	Run dispatcher job.	Consumer log records SUCCESS.
EVT-QA-006	Dead Letter	Force consumer failure beyond retries.	event_dead_letter row created and visible in UI.
EVT-QA-007	Correlation	Run recommendation-policy-approval-action flow.	All events share same correlation_id.
EVT-QA-008	Timeline	Open subject timeline for AI Agent.	Events are chronological and complete.
EVT-QA-009	Security	User from another tenant attempts event read.	Access denied / no data returned.
EVT-QA-010	Payload Masking	Unauthorized user views restricted event.	Sensitive fields masked.
EVT-QA-011	Immutability	Attempt update/delete of event through app path.	Operation blocked.
EVT-QA-012	Audit Export	Generate export for date range.	Export log created with hash and event count.
EVT-QA-013	Performance	Search events with date range and filters.	Response within agreed threshold.
EVT-QA-014	Retry	Retry dead-letter event.	Retry action is audited and status changes.
EVT-QA-015	Integration	Relationship revocation emits event.	Timeline shows RELATIONSHIP_REVOKED.

16.2 Acceptance Criteria

- All major state-changing operations generate governance events.
- Events follow canonical envelope and pass validation.
- Events are append-only and include tenant_id, actor, subject, timestamp, correlation_id, classification and retention_class.
- Event search APIs support filters required by frontend screens.
- Subject and correlation timelines can be reconstructed from events.
- Outbox dispatcher and dead-letter review work for failed processing.
- Event visibility respects RBAC, ABAC, tenant boundaries and classification.

- Audit export produces a package record and export hash.
- At least one full lifecycle demo is complete: recommendation -> policy evaluation -> approval -> action -> audit timeline.

17. Deliverables and Definition of Done

Deliverable	Description	Owner	Done When
Phase 4 Data Model Scripts	DDL, indexes, seed data and migration scripts.	Aayush	Scripts run cleanly and tables are verified.
Event Backend Module	Models, schemas, repository, publisher, dispatcher and query services.	Pankaj	Unit and integration tests pass.
Event API Specification	OpenAPI-style contract and request/response examples.	Aayush / Pankaj	Frontend and QA can implement without clarification.
Frontend Event Screens	Event Explorer, Detail, Timeline, Correlation, Dead Letter, Audit Export.	Poorvith	Screens integrated with APIs and tested.
Audit Timeline Flow	Subject and correlation lifecycle reconstruction.	Pankaj / Poorvith	Timeline demo works end to end.
Security Controls	RBAC/ABAC and payload masking.	Pankaj	Security tests pass.
QA Evidence Pack	Test cases, screenshots/API responses, defect log.	Pankaj	Critical tests complete and defects closed.
Handover Pack	Scripts, API docs, routes, acceptance checklist and known limitations.	Project Manager	Team sign-off captured.

18. Appendices

18.1 Starter Event Catalogue

Category	Event Type	Description
Identity	USER_CREATED	User profile created.
Identity	ROLE_ASSIGNED	Role granted to user.
Identity	DELEGATION_STARTED	Delegated authority begins.
Registry	AI_MODEL_REGISTERED	AI model added to registry.
Registry	AI_MODEL_UPDATED	AI model metadata updated.
Registry	AGENT_REGISTERED	AI agent registered.
Registry	AGENT_PERMISSION_UPDATED	Agent permission changed.
Registry	TOOL_REGISTERED	Tool/API/connector registered.
Policy	POLICY_CREATED	Policy created.
Policy	POLICY_ACTIVATED	Policy activated.
Policy	POLICY_EVALUATED	Policy evaluation completed.
Policy	POLICY_TRIGGERED	Policy condition triggered.
Risk	RISK_EVALUATED	Risk score calculated.
Risk	RISK_THRESHOLD_EXCEEDED	Risk threshold breached.
Recommendation	RECOMMENDATION_CREATED	Recommendation created.
Recommendation	RECOMMENDATION_EVALUATED	Recommendation evaluated by policies.
Approval	APPROVAL_REQUESTED	Approval request created.
Approval	APPROVAL_GRANTED	Approval granted.
Approval	APPROVAL_REJECTED	Approval rejected.
Escalation	ESCALATION_TRIGGERED	Escalation created.
Action	ACTION_CREATED	Action created from approved decision.
Action	ACTION_EXECUTED	Action completed/executed.
Action	ACTION_FAILED	Action failed.
Agent	AGENT_ACTION_REQUESTED	Agent requested action.
Agent	AGENT_ACTION_BLOCKED	Agent action blocked by control.
Agent	TOOL_ACCESS_ATTEMPTED	Tool access attempted.
Violation	POLICY_VIOLATION_DETECTED	Policy violation detected.
Violation	SLA_BREACHED	SLA breached.
Evidence	EVIDENCE_ATTACHED	Evidence linked to object.
Evidence	EXPLANATION_CAPTURED	AI explanation captured.
Audit	AUDIT_TIMELINE_VIEWED	Audit timeline viewed.
Audit	AUDIT_PACKAGE_EXPORTED	Audit package exported.
Audit	EVENT_CORRECTION_RECORDED	Correction event recorded.

18.2 Glossary

Term	Meaning
Event	Immutable record that something meaningful happened.
Event Producer	Service or system that creates an event.
Event Consumer	Service or system that reacts to an event.

Subject	Primary object the event is about.
Actor	Human, AI agent, system or service account responsible for the event.
Correlation ID	Identifier linking all events in one business flow.
Causation ID	Identifier of the event/action that caused this event.
Outbox	Durable queue table used to reliably dispatch events after persistence.
Dead Letter	Failed event processing record requiring review or retry.
Audit Timeline	Chronological reconstruction of subject lifecycle from events.
Retention Class	Policy class determining how long an event should be retained.
Event Hash	Integrity value used to detect tampering or support audit exports.

18.3 Source Alignment

This implementation specification is derived from the GuardianIQ Step 4 - Governance Event Architecture foundation document. It preserves the foundation's core principle that every recommendation, approval, escalation, override, violation, agent action and audit update must be treated as a governed event.